

SIGNVERSE

Complete Project Documentation

Version: 1.0

Date: March 2026

Table of Contents

1. Project Overview
 2. Problem Statement
 3. Objectives
 4. System Architecture
 5. Working Flow / Execution Flow
 6. Features
 7. Technology Stack
 8. Folder Structure Explanation
 9. Installation & Setup Instructions
 10. Configuration Details
 11. API Endpoints
 12. Model Details & Training Pipeline
 13. Frontend Architecture
 14. Deployment Process
 15. Future Enhancements
 16. Conclusion
-

1. Project Overview

SignVerse is a real-time, AI-powered sign language detection and translation system that bridges the communication gap between hearing-impaired individuals and the general public. It supports two distinct sign languages — **American Sign Language (ASL)** and **Indian Sign Language (ISL)** — within a single unified web application.

The system captures live webcam video, processes hand gestures using **Google MediaPipe** for landmark detection, classifies the gestures using machine learning models (**Random Forest for ASL** and **Deep Neural Network for ISL**), and builds readable sentences character by character.

Users can then speak the constructed sentence aloud using built-in **Text-to-Speech (TTS)** or correct it using **Google Gemini AI** for intelligent auto-correction.

2. Problem Statement

According to the **World Health Organization**, over **466 million people worldwide** have disabling hearing loss.

Sign language is the primary mode of communication for the deaf community, yet the vast majority of the hearing population cannot understand it. This creates a significant barrier in everyday interactions — from **healthcare consultations to job interviews**.

Existing solutions typically:

- Support only a single sign language (usually ASL)
- Require specialized hardware such as gloves or depth cameras
- Lack real-time processing capability
- Do not provide sentence-level output

There is therefore a critical need for a **low-cost, real-time, multi-language sign language detection system** that requires only a standard webcam and produces readable text output.

3. Objectives

Real-Time Detection

Classify hand gestures into corresponding alphabets or numbers with minimal latency using a standard webcam.

Dual-Language Support

Support both **ASL (26 letters + space)** and **ISL (26 letters + 9 digits)** within a single application, switchable instantly.

Sentence Construction

Accumulate detected characters into **words and sentences** using a built-in sentence builder.

AI-Powered Correction

Integrate **Google Gemini AI** to auto-correct noisy gesture-typed text into meaningful words and sentences.

Text-to-Speech Output

Convert the accumulated sentence into spoken audio using **pyttsx3**, enabling audible communication.

Accessible & Low Cost

Run on any machine with **Python and a webcam** without specialized hardware.

Premium User Interface

Provide a **modern glassmorphism themed UI** that is responsive and intuitive.

4. System Architecture

High-Level Architecture

USER / BROWSER

SignVerse Web Interface

- Mode Switcher
- Live Video Feed (MJPEG)
- Controls (Speak, Clear, Space, Delete)
- Sentence Builder Panel

HTTP Requests / Polling

FLASK BACKEND (app.py)

Components:

ASL Pipeline

- MediaPipe Landmarker (1 hand)
- Random Forest Classifier

ISL Pipeline

- MediaPipe Landmarker (2 hands)
- Deep Neural Network (Keras)

AI Services

- Gemini AI (Text Correction)

Speech System

- TTS Engine (pyttsx3)

Video System

- OpenCV VideoCapture (Webcam Interface)
-

Component Breakdown

Component	Role
Flask Web Server	Serves UI and handles REST APIs
MediaPipe HandLandmarker	Detects 21 hand landmarks
ASL Classifier	Random Forest model for ASL letters
ISL Classifier	Deep Neural Network model for ISL characters
Gemini AI Client	Corrects gesture-typed text
TTS Engine	Converts sentence into speech
Frontend	Glassmorphism web UI

5. Working Flow / Execution Flow

System Execution Flow

1. User opens the web browser.
 2. index.html loads and JavaScript initializes.
 3. startStream() calls **GET /video_feed**.
 4. Flask opens the webcam using **OpenCV**.
 5. Frames are captured from the webcam.
 6. **MediaPipe** detects hand landmarks.
 7. Landmarks are normalized.
 8. ML model predicts the character.
 9. Stability and cooldown checks occur.
 10. Character is appended to the sentence.
 11. Frame overlays are drawn.
 12. Frame is encoded as JPEG and streamed to browser.
-

Frontend Polling Flow

Every **250ms**:

Frontend calls **GET /get_data**

Response includes:

- Current prediction
- Current sentence
- Active mode (ASL / ISL)

The UI updates the detected character and sentence display.

AI Correction Flow

When the user clicks **Correct with AI**:

1. Frontend sends **POST /correct**
 2. Backend reads the accumulated sentence
 3. Sends text to **Gemini 2.0 Flash**
 4. Gemini returns corrected sentence
 5. Backend replaces the sentence
 6. Frontend updates the display
-

6. Features

Core Features

Feature	Description
Dual Mode Detection	Switch between ASL and ISL instantly
Real-Time Processing	30 FPS webcam processing
Sentence Builder	Characters accumulate into sentences
AI Text Correction	Gemini corrects noisy text
Text-to-Speech	pyttsx3 reads sentences aloud
Prediction Stability	ISL uses 5 frame stability buffer
Cooldown Mechanism	Prevents repeated character insertion

UI Features

Feature	Description
Glassmorphism Theme	Frosted glass UI cards
Mode Switcher	Toggle between ASL and ISL
Live Status Indicator	Shows stream status
Gesture Reference Guide	Visual sign charts
System Status Dashboard	Shows model load status
Toast Notifications	User feedback messages
Responsive Layout	Works on desktop and mobile

ISL Special Gestures

Gesture Action	
1	Add space
2	Delete last word

7. Technology Stack

Backend

Technology	Purpose
Python	Core programming language
Flask	Web framework
Flask-CORS	Cross origin support
OpenCV	Webcam capture
MediaPipe	Hand landmark detection
TensorFlow / Keras	ISL neural network
Scikit-learn	ASL Random Forest model
NumPy	Numerical processing
pyttsx3	Offline text-to-speech
google-genai	Gemini AI integration

Frontend

HTML5 — Structure
CSS3 — Glassmorphism UI
JavaScript (ES6) — Client logic

AI Models

Model	Algorithm	Features	Classes
ASL Model	Random Forest	42	26
ISL Model	Deep Neural Network	84	35
MediaPipe	Hand Landmark Detection	21 points	Pre-trained

8. Folder Structure

Unified_Sign_Language/

app.py

generate_keypoints.py

train_model.py

templates/

index.html

static/

script.js

style.css

images/

asl_signs.jpeg

isl_gestures.png

models/

asl_model.p

isl_model.h5

isl_label_classes.json

hand_landmarker.task

environment.yml

requirements.txt

run.bat

README.md

.gitignore

9. Installation & Setup Instructions

Prerequisites

Operating System: Windows / Linux / macOS
Python: 3.10
Webcam required
Conda recommended

Method 1 — Conda (Recommended)

Create environment

```
conda env create -f environment.yml
```

Activate environment

```
conda activate sign_language_unified
```

Install Gemini SDK

```
pip install google-genai
```

Set API key

Windows

```
set GEMINI_API_KEY=your_key
```

Linux/Mac

```
export GEMINI_API_KEY=your_key
```

Run application

```
python app.py
```

Open browser

```
http://localhost:5050
```

10. Configuration Details

Environment Variables

Variable	Description
GEMINI_API_KEY	Gemini API key
TF_CPP_MIN_LOG_LEVEL	Suppress TensorFlow logs
TF_USE_LEGACY_KERAS	Enable legacy compatibility

Application Constants

Constant	Value
ASL Cooldown	3 seconds
ISL Cooldown	1 second
Stability Threshold	5 frames
Camera Resolution	1280x720
FPS	30
Polling Interval	250 ms
Server Port	5050

11. API Endpoints

GET Endpoints

Endpoint	Description
/	Main UI page
/video_feed	MJPEG video stream
/get_data	Returns prediction + sentence
/model_status	Model status information

POST Endpoints

Endpoint	Function
/switch_mode	Switch ASL / ISL
/clear_sentence	Clear sentence
/speak	Speak sentence
/stop	Stop stream
/delete_last	Delete character
/add_space	Add space
/correct	Gemini text correction

12. Model Details

ASL Model

Algorithm: Random Forest

Library: Scikit-learn

Features:

21 landmarks $\times$ 2 coordinates = **42 features**

Classes:

A–Z + Space

Model file:

asl_model.p

ISL Model

Architecture:

Dense(128)

BatchNorm

Dropout

Dense(64)

BatchNorm

Dropout

Dense(32)

Softmax Output

Features:

84 (2 hands)

Classes:

1–9 + A–Z

Model file:

isl_model.h5

13. Frontend Architecture

Main layout components:

Navigation bar

Hero section

Mode switcher

Video feed panel

Sentence builder

Controls panel

Gesture reference guide

System status dashboard

JavaScript Functions

switchMode()
startStream()
stopStream()
restartStream()
fetchData()
clearSentence()
speakText()
deleteLast()
addSpace()
correctWithAI()
fetchModelStatus()

14. Deployment Process

Run locally:

```
python app.py
```

Production deployment:

Use **Waitress**

```
pip install waitress
```

```
waitress-serve --host=0.0.0.0 --port=5050 app:app
```

Use **Nginx reverse proxy** for HTTPS.

Docker example:

```
FROM python:3.10-slim
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN pip install -r requirements.txt
```

```
CMD ["python","app.py"]
```

15. Future Enhancements

1. Word-level sign detection
2. WebRTC streaming
3. Predictive text suggestions
4. User authentication
5. Conversation history
6. More sign languages (BSL, JSL)
7. Mobile app with TensorFlow Lite

8. Face and pose recognition
 9. Multilingual TTS (Hindi, Marathi)
 10. Dataset contribution system
-

16. Conclusion

SignVerse demonstrates the practical application of **computer vision and machine learning for accessibility**.

The system successfully combines:

- Real-time gesture detection
- Dual sign language support
- Machine learning classification
- AI-based text correction
- Text-to-speech communication

Key technical highlights include:

- Dual-model architecture with runtime mode switching
- MediaPipe hand landmark detection (21 keypoints per hand)
- Intelligent sentence building with stability buffers
- AI correction using Gemini
- Deployment with only a webcam and Python environment

This project showcases strong skills in:

- **Full-stack development (Flask + Web UI)**
- **Machine learning (Random Forest + Deep Neural Networks)**
- **Computer vision (MediaPipe + OpenCV)**
- **AI integration (Gemini API)**

SignVerse represents a meaningful step toward **accessible communication technology for the deaf and hard-of-hearing community**.
